

AMiT
Arba Minch Institute of Technology
አርባ ምንጭ ቴክኖሎጂ ኢንስቲትዩት
Arba Minch University
አርባ ምንጭ ዩኒቨርሲቲ

Faculty of Computing and Software Engineering

Department of Software Engineering

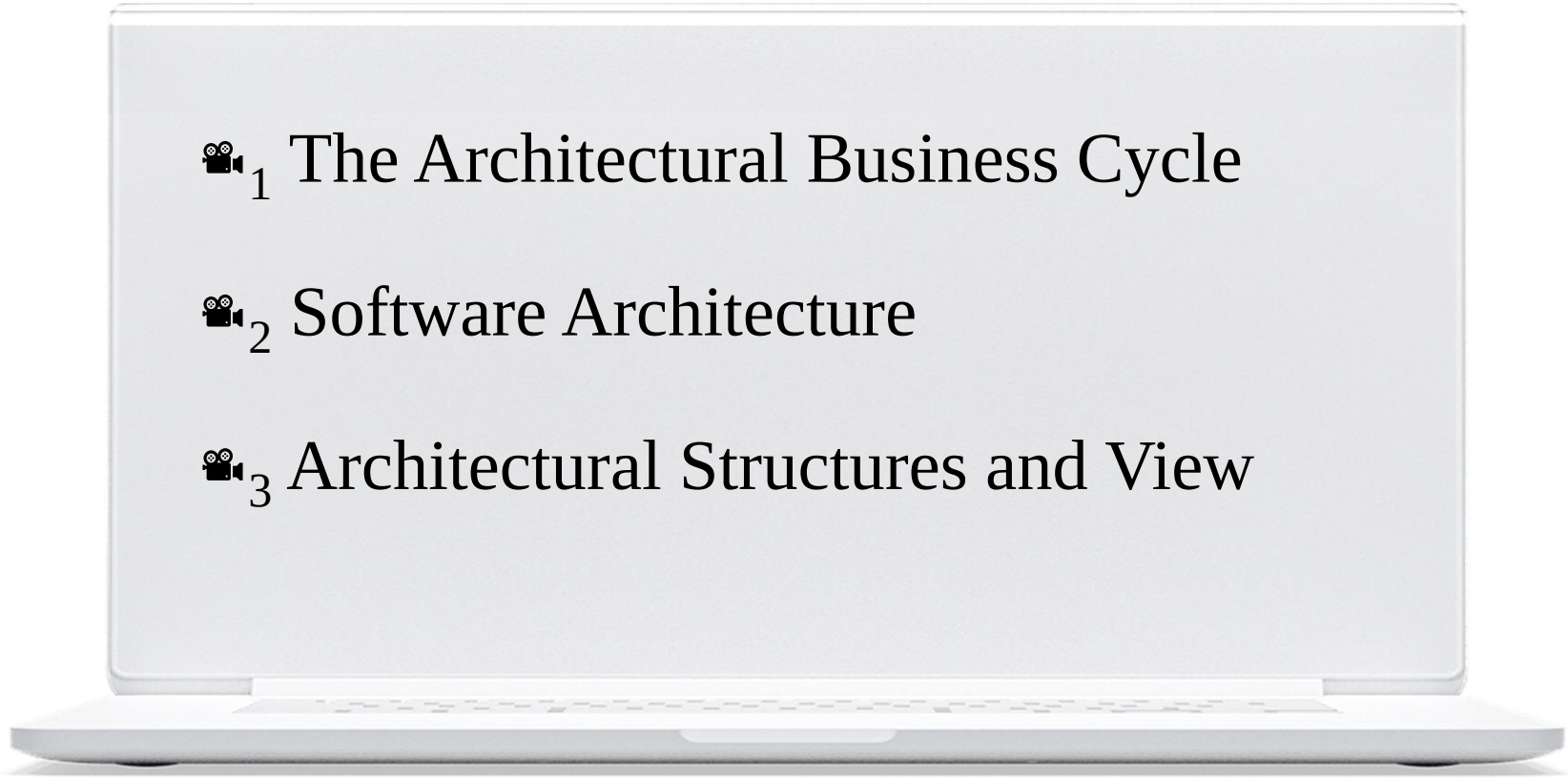
SENG5232- Software Architecture and Design

Chapter 3: Envisioning Architecture

"One thing most software architects fail to realize is that a software architect is also a leader."

Richard Monson-Haefe

Contents


1 The Architectural Business Cycle
2 Software Architecture
3 Architectural Structures and View

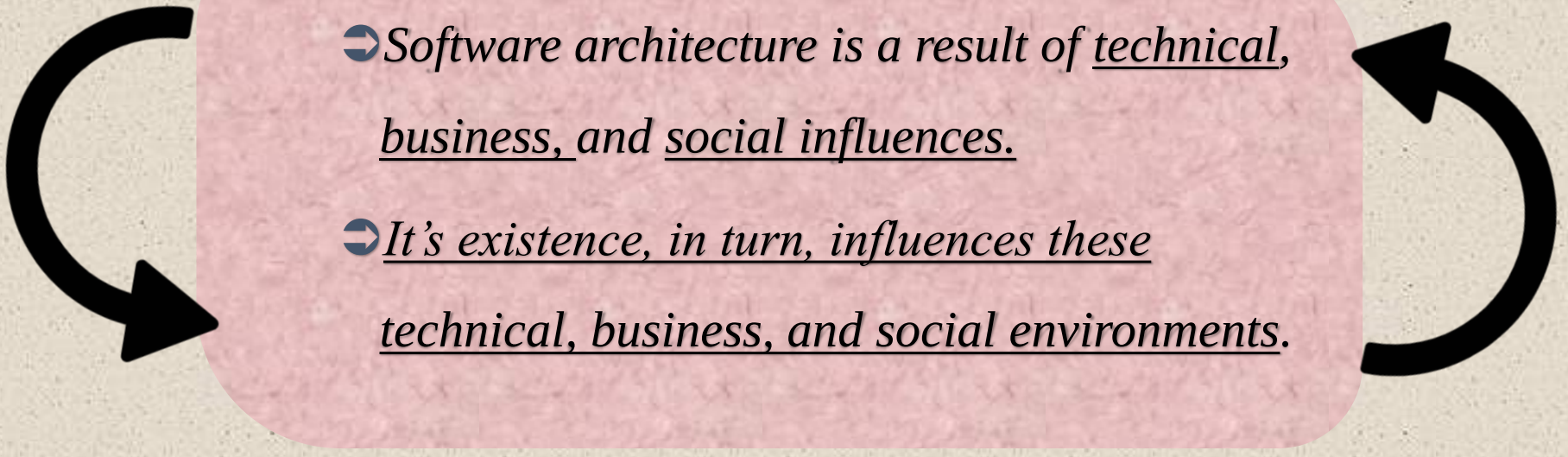
Brainstorming

BS1: Can you define Software Architecture?

BS2: Can you explain what by meant the Architecture Business Cycle ?

BS3: Can you define what by meant the Architecture view ?

The Architecture Business Cycle (ABC)



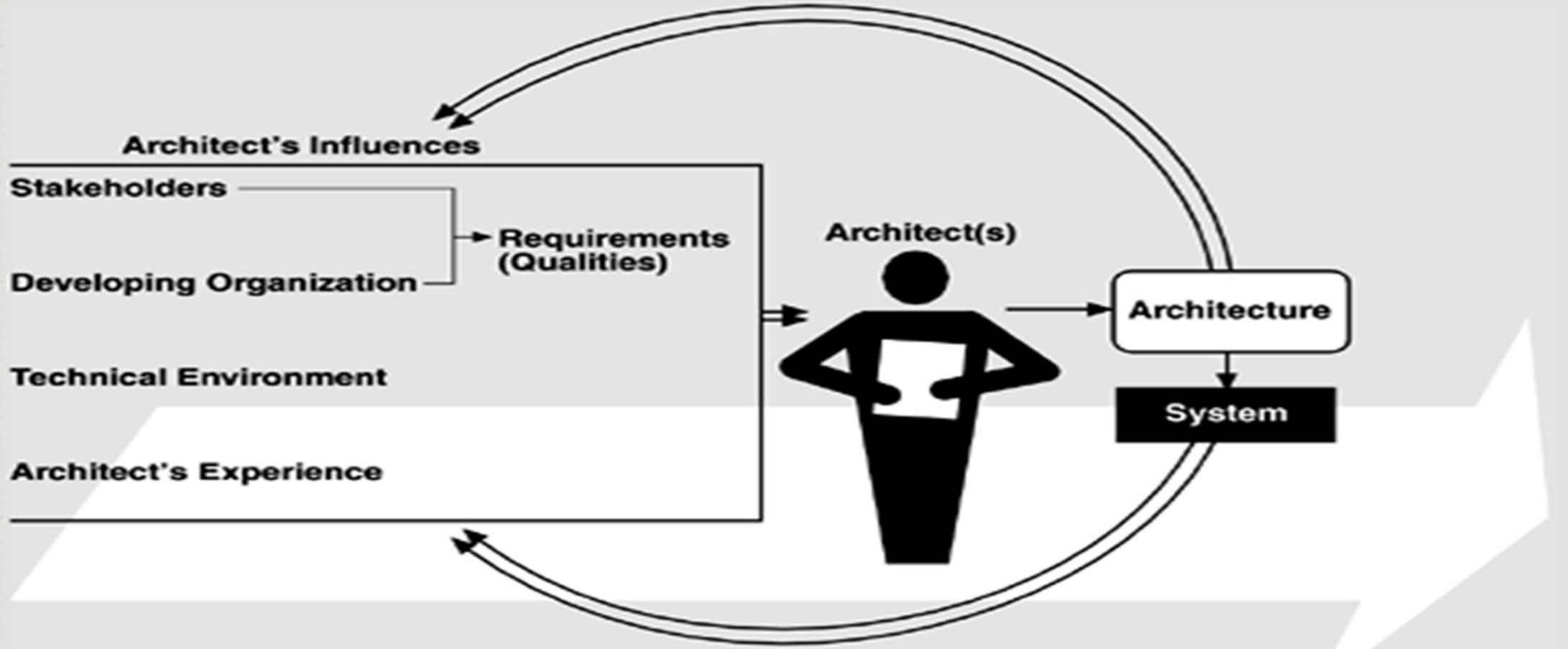
- *Software architecture is a result of technical, business, and social influences.*
- *It's existence, in turn, influences these technical, business, and social environments.*

➥ *This cycle of influences from the environment to the architecture and back to the environment is called the* ***Architecture Business Cycle.***

Conti..

- **Software Architecture:** refers to the **high-level structure** and **organization** of a software system. It encompasses ***the key components, their relationships, and the overall design decisions*** that guide the development of the software.
- **Technical Influences:** These are the technological factors that affect software development. This could include the ***programming languages, frameworks, databases, and hardware*** used in the system. Technical factors can also include considerations like ***performance, scalability, security, and maintainability.***
- **Business Influences:** Business factors encompass the ***objectives***, ***goals***, *and* ***constraints*** related to the software project. It includes considerations like ***budget, deadlines, market demand, competition, and revenue generation***. Business influences also involve decisions about how the software aligns with the overall business strategy.
- **Social Influences:** Social factors relate to the human element in software development. This includes the ***skills and expertise of the development team, the company culture, and how stakeholders, users, and customers perceive and interact with the software***. Social influences can also involve legal and ethical considerations.

The Architecture Business Cycle (ABC)



Where Do Architectures Come From?

- Architectures are influenced by:
  - » System stakeholders.
  - » The developing organization.
  - » The background and experience of the architects.
  - » The technical environment.

Influence of system stakeholders on Architect



Developing organization management stakeholder

Low Cost

Marketing stakeholder

Neat features, Short time to market, Low Cost

End User Stakeholder

Performance, Security , Reliability, Usability

Maintenance organization stakeholder

Modifiability

Customer stakeholder

Low cost, Timely delivery, not changed often



Influence of system stakeholders on Architect

- *Acceptable system involves properties* such as *performance*, *reliability*, *availability*, *memory utilization*, *security*, *modifiability*, *usability*, *interoperability with other system*.
- The underlying problem, is that each stakeholder has *different concerns* and *goals*, some of which may be contradictory.

✓ These concerns and goals can vary widely and sometimes even conflict with one another.
For example:
- Users may prioritize usability and ease of use.
- Business owners may focus on cost-effectiveness and revenue generation.
- Developers may be concerned with maintainability and technical feasibility.
- Regulators may emphasize compliance with legal and security standards.

➪ The reality is that the architect often has to fill in the blanks and mediate the conflicts.

Influence of developing organization on architect

- Architecture is also influenced by the structure and nature of organization.
- There are three classes of influence that come from the developing organization
  - Immediate business(short term).
  - Long-term business.
  - Organizational structure(subcontract).

Conti..

- **Immediate Business (Short-term):** It pertains to short-term or immediate business concerns and objectives. It suggests that the ***organization's current priorities and short-term goals*** will shape the software architecture.
    - For example, if the organization's short-term goal is to release a product quickly to capture a market opportunity, the architecture might prioritize speed and time-to-market over long-term scalability or maintainability.
- **Long-term Business:** It refer to an ***organization's strategic goals and objectives*** that extend beyond the immediate future. The architecture of a software system needs to align with the organization's long-term vision.
    - This may involve considerations such as the ability to adapt to changing market conditions, evolving regulatory requirements, and accommodating future growth. An architecture that is overly tailored to short-term goals may not be suitable for long-term sustainability.
- **Organizational Structure (Subcontract):** The structure of the organization, including ***how work is divided and delegated,*** can significantly influence software architecture.
    - If an organization relies on subcontracting or outsourcing, the architectural decisions may need to consider how different components or subsystems are developed and integrated. The organization's choice of partners and collaborators can also impact architectural decisions, such as the use of standardized interfaces for interoperability.

Influence of the background and experience of architect

- *Architects for a system have had good results using a particular architectural approach*, chances are that they will try that same approach on a new development effort, or may be reluctant to try it again.
- *Architectural choices may also come from an architect's education and training, exposure to successful architectural patterns*, or exposure to systems that have worked particularly poorly or particularly well.

Influence of technical environment on architect

- Architect's background and experience is reflected by the **technical environment.**
  - The ***technical environment*** refers to the set of tools, technologies, methodologies, and constraints that exist within a given software development project or organization.
    - ***Past Experiences, Skills and Expertise, Knowledge Base, Industry Trends, Organizational Culture, Constraints and Requirements and etc.***
- **Current environment** will influence the architecture standard industry practices or software engineering techniques.
  - Current environment referring to the idea that the context in which software development takes place—characterized by the tools, technologies, methodologies, and trends—plays a significant role in shaping how software architects design and build systems.
    - ***Technology Adoption, Methodology and Processes, Industry Trends, Best Practices, Regulatory and Compliance Requirements, Emerging Technologies, Community and Collaboration, Evolving Software Engineering Practices and etc.***

Ramifications of the Various Influences on an Architecture

- The properties required by the business and organizational goals are seldom completely understood.
- Customer requirements are often not complete or well documented.
- Architects must identify and actively engage stakeholders to solicit their needs and expectations.
- In addition to technical skills, architects need to be skilled in diplomacy, negotiation, and communication.

Architectures Affect the Factors that Influence Them

- It affects the structure of the developing organization.
- It can affect the goals of the developing organization.
- It can affect customer requirements for the next system.
- It will affect the architectures of future systems.
- It may affect the software engineering culture.

Architecture Process

Software Architecture Activities

- Creating the business case for the system.
- Understanding the requirements.
- Creating or selecting the architecture.
- Communicating the architecture.
- Analyzing or evaluating the architecture.
- Implementing based on the architecture.
- Ensuring conformance to an architecture.

Software Architecture Activities

✓ *Creating the business case for the system*

- **Creating a business case** involves laying out the fundamental components of a business idea, such as the value proposition, target market, revenue model, cost structure, and financial projections.
- **Understanding the needs of market** involves market research, which can be a multifaceted process that includes studying customer behavior, analyzing market trends, identifying pain points, and assessing competition.

- It is simple to create a business case than understanding the needs of market.
  - How much should be the product cost?
  - What is the targeted market?
  - What is the targeted time to market?
  - Will it need to interface other system?
  - Are there system limitations ?

Software Architecture Activities

✓ Understanding the requirements

- There are variety of techniques to understand requirements from stakeholders.
  - **Object oriented analysis (use cases & scenarios)**: it define system functionalities and interactions from the user's perspective.
  - **Safety Critical Systems:**
    - **Finite state machine models:** Ensure predictability and reliability by representing system states and transitions.
    - **Formal specification languages:** Use mathematical languages for precise system behavior specification and analysis.
  - **Quality attributes and prototypes:**
    - **Quality Attributes:** Specify non-functional requirements like performance, reliability, and security.
    - **Prototypes**: Create models to visualize system behavior, interfaces, and usability.

Software Architecture Activities

Creating or selecting the architecture

- **Conceptual integrity:** the coherence and consistency of the architectural design, where all components and elements fit together seamlessly to form a unified and well-structured whole.
- A small number of minds coming together to design the system's architecture.

Communicating the architecture

For effective architecture

- It must be communicated clearly and unambiguously to all stakeholders.
- Developers must understand work assignments.
- Testers must understand the task structures.
- Management must understand the scheduling implications.

Software Architecture Activities

Analyzing or Evaluating the architecture

- Out of multiple designs, after analysing, some design will be accepted or some are rejected.
- Evaluating an architecture for the qualities.
  - It's supports is essential to ensure the stakeholders satisfaction (needs).
  - Scenario-based techniques are used for evaluation of architecture.

Common methods:

- **Architecture Tradeoff Analysis Model (ATAM):** a method for evaluating and analyzing the tradeoffs between different architectural design options to make informed decisions.
- **Cost Benefit Analysis Model (CBAM):** a technique for comparing the costs and benefits of various design choices to determine the most economically advantageous option.

Software Architecture Activities

Implementing based on the architecture

- Concerned with keeping the developers faithful to the structures.
- Should have an environment that assists developers in creating the architecture.

Ensuring conformance to an architecture

- Finally, when an architecture is created and used, it goes into maintenance phase.
- Constant vigilance is required to ensure that actual architecture and its implementations remain faithful to each other.

What Makes a Good Architectural Process?

» It should be the product of a single architect or small group of architects.

» The architect should gather both the functional requirements and a well specified, prioritized list of quality attributes.

» It should be well documented, with at least one static view and one dynamic view.

» It should be reviewed by the system's stakeholders.

What Makes a Good Architectural Process? (Cont'd)

» The architecture should be analyzed for applicable quantitative measures (e.g., maximum throughput) and formally evaluated for quality measures.

» A "skeletal" system should be developed to facilitate incremental implementation.

» Resource contention areas should be minimized and clearly identified.

What Makes a Good Architectural Structure? (Cont'd)

» The architecture should feature ***well-defined modules*** based on the principles of information hiding and separation of concerns.

» Each module should have a well-defined interface that hides changeable aspects.

» Quality attributes should be achieved using well-known architectural tactics.

» The architecture should be ***independent*** of particular versions of commercial products or tools.

What Makes a Good Architectural Structure? (Cont'd)

» Modules that produce data should be ***separate*** from modules that consume data.

» For parallel-processing systems, the architecture should feature well-defined processes or tasks that do not necessarily mirror the module decomposition structure.

» Every task or process should be specified so that its assignment to a particular processor can be ***easily changed.***

» It should contain a small number of simple interaction patterns.

Software Architecture

- The **software architecture** is the structure of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them.

- It involves the high level structure of software system abstraction, by using decomposition and composition, with architectural style and quality attributes.
- It must describe its group of components, their connections, interactions among them and deployment configuration of all components.
- It must conform to the *major functionality and performance requirements* of the system, as well as satisfy the ***non-functional requirements*** such as reliability, scalability, portability, and availability.

Software Architecture(Cont'd)

A software architecture can be defined in many ways −

1. **UML (Unified Modeling Language)** − UML is one of object-oriented solutions used in software modeling and design.
2. **Architecture View Model (4+1 view model)** − Architecture view model represents the functional and non-functional requirements of software application.
3. **ADL (Architecture Description Language)** − ADL defines the software architecture formally and semantically.

Architectural Patterns, Reference Models & Reference

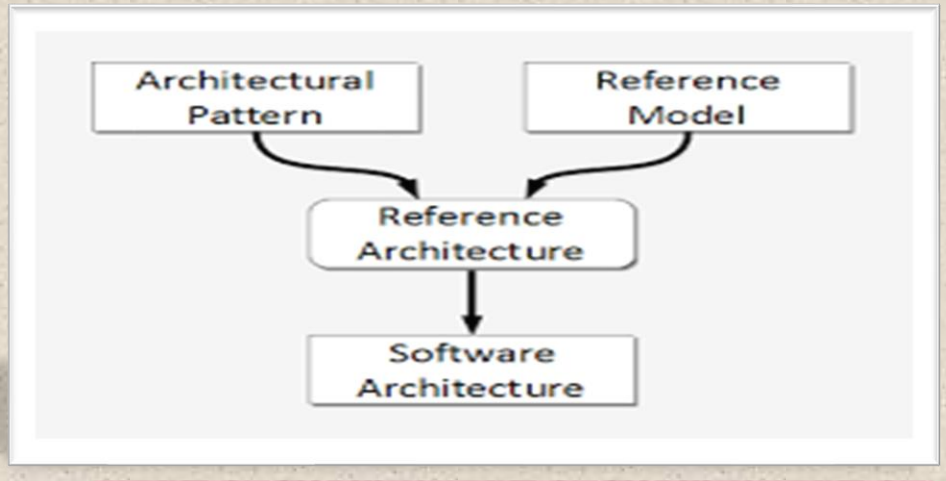
- **Architectural Pattern:** is a reusable and proven design solution that addresses recurring architectural design problems. It provides a general structure or blueprint for organizing and designing software systems to address common architectural challenges.
  - **Examples,** Model-View-Controller (MVC) pattern, the Layered architecture pattern, and the Microservices pattern.
- **Reference Model:** is a conceptual framework or blueprint that defines the key components, relationships, and interactions in a particular domain, often used for standardization and communication purposes. It provide a high-level, abstract representation of a system or technology, focusing on defining concepts and relationships without specifying implementation details.
  - **Example**, the OSI (Open Systems Interconnection) model is a reference model for computer networking, and it defines seven layers for network communication.
- **Reference Architecture:** is a specialized type of reference model that provides a more concrete and detailed architectural framework for a particular domain or industry. It combines the principles of architectural patterns with specific design guidelines and best practices for a given application area.
  - Reference architectures are intended to guide the design and implementation of systems within a specific context, ensuring alignment with industry standards and goals. They are often used as a starting point for designing systems within a specific domain, industry, or organization.

Conti..

These terminology concepts:

Architectural Patterns, Reference Models & Reference Architectures

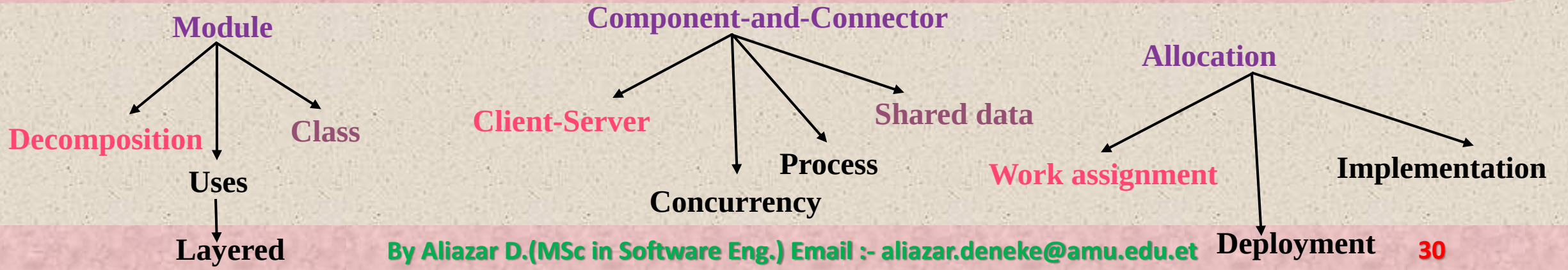
- *They are not architectures rather they are useful concepts that capture elements of an architecture.*
- *Each of them are outcome of early design decisions.*



*Therefore a **software architect** must design a system that provides concurrency, portability, modifiability, usability, security, and the like, and that reflects consideration of the tradeoffs among these needs.*

Architectural Structures and views

- *It can be largely divided into three groups, depending on the broad nature of the elements they show.*
- 1. **Module structures** represent the organization of a system's components into distinct, encapsulated modules, helping manage complexity and promote modularity in software design.
- 2. **Component-and-connector structures** describe how a system's components interact and communicate with each other, illustrating the flow of data, control, and dependencies within the system.
- 3. **Allocation structures** specify how system elements are assigned or allocated to hardware, software, or other resources, ensuring efficient resource utilization in a system's architecture.
  - Each of these structures provides a different perspective and design handle on a system, and each is valid and useful in its own right. In general, mappings between structures are many to many.
  - **Individual structures** bring with them the power to manipulate ***one or more quality attributes***. They represent a powerful separation-of- concerns approach for creating the architecture.



Architectural View(Cont'd)

- **The architectural view** of a system is abstract, distilling away details of implementation, algorithm, and data representation and concentrating on the *behavior and interaction of "black box" elements*.
- **An architectural view** provides a specific perspective or representation of a system's structure and behavior, aiding in the understanding and communication of its design.

4+1 Architectural Views

- Architectural views are the ***perspectives that different stakeholders*** have about the architecture of the system.
- And also it is about the notations the architect used to represent the architecture to these stakeholders.
- The ***whole architecture*** of the system ***can't be represented with one model***, as each model represents only one view of the system and the views required are many. As a result, an architect needs to have more than one views.
- The most widely used views are 4 and others add one extra, hence the name 4+1 architecture.
- Each view addresses a specific set of concerns and is designed to facilitate communication among stakeholders.

4+1 Architectural Views

The 4+1 views of an architecture are:

1. **Logical *view:*** defines the high-level structure of a software system, emphasizing its functional components and their relationships.
   - It shows key abstractions of the system as objects or object classes.
   - It relates functional requirements with entities.
2. **Process *view:*** focuses on the dynamic aspects of a system, illustrating how processes and tasks interact to achieve desired functionality.
   - It shows how, at run-time, the system is composed of interacting processes.
   - It judges non-functional requirements of the system.
   - Is all about the synchronization and concurrency of processes of the system.
3. **Development view:** details the organization of source code, modules, and development activities to support software development and maintenance.
   - It shows the breakdown of the software into components for development, important for managers and developers.
4. **Physical view:** describes the system's deployment perspective ensuring scalability, performance, and resource allocation.
   - It shows the system hardware and how software components are distributed across the processors in the system, important for system engineers.
   - Is all about mapping of the software onto the hardware and its distribution.

4+1 Architectural Views(Cont'd)

Kruchten's 4+1 View Model :

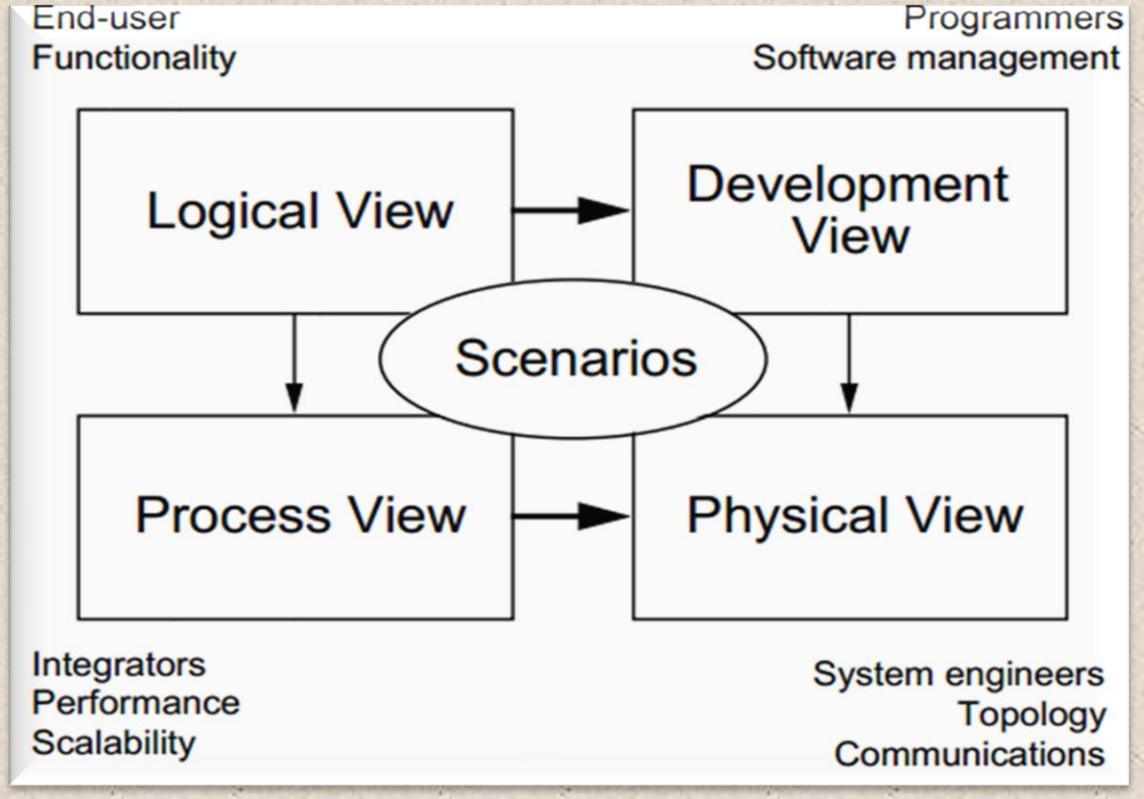
4+1 is a view model designed by Philippe Kruchten for "describing the architecture of software-intensive systems, based on the use of multiple, concurrent views". The views are used to describe the system from the viewpoint of different stakeholders, such as end-users, developers and project managers. The four views of the model are logical, development, process and physical view. In addition selected use cases or scenarios are used to illustrate the architecture serving as the 'plus one' view. Hence the model contains 4+1 views.



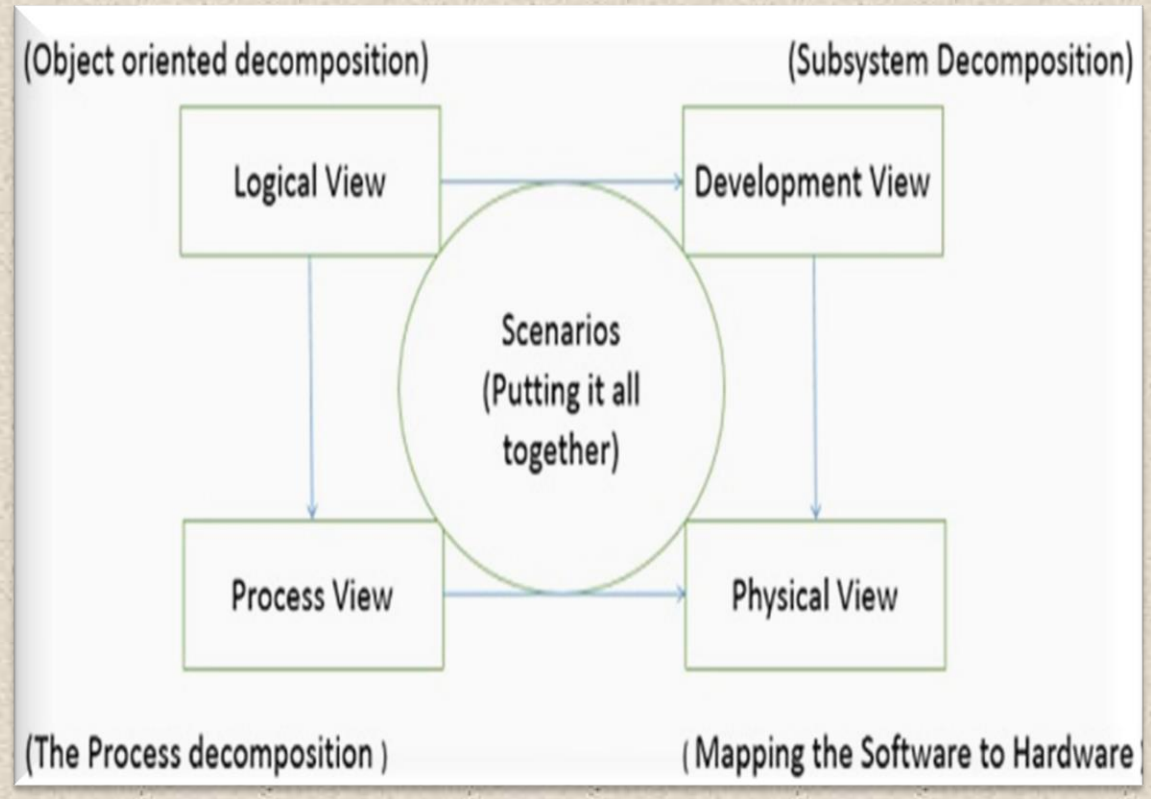
Philippe Kruchten is a Canadian software engineer, and Professor of Software Engineering at University of British Columbia in Vancouver, Canada, known as Director of Process Development at Rational Software, and developer of the 4+1 Architectural View Model.

- *Kruchten's four views follow:*
  - ***Logical**. The elements are "key abstractions," which are manifested in the object-oriented world as objects or object classes.*
    - *This is a **module view. i.e. Class, State, Object, Sequence, Communication Diagram.***
  - ***Process**. This view addresses concurrency and distribution of functionality.*
    - *It is a **component-and-connector view**. i.e. **Activity Diagram.***
  - **Development**. This view shows the organization of software modules, libraries, subsystems, and units of development.
    - It is an ***allocation view***, mapping software to the development environment. i.e. ***Component, Package diagram.***
  - **Physical**. This view maps other elements onto processing and communication nodes.
    - It is also an allocation view. i.e. ***Deployment diagram.***

Architectural View(Cont'd)



or



Scenarios: The description of an architecture is illustrated using a small set of use cases, or scenarios, which become a fifth view. The scenarios describe sequences of interactions between objects and between processes. They are used to identify architectural elements and to illustrate and validate the architecture design. They also serve as a starting point for tests of an architecture prototype. This view is also known as the use case view.

Architectural View(Cont'd)

Correspondence between views

➲ **From logical view to process view**

- Different characters of classes in logical view are:
  - ✓ **Activeness/passiveness**
    - **Active class** invokes its own or other objects' operations and has full control over invocations to its operations.
    - **Passive class** can't invoke any operation and has no control over invocations to its operations.
    - ***Protected class*** can't invoke any operations but has arbitration to its own operations' invocations.

➲ **From logical view to process view**

- Different characters of classes in logical view are:
  - **Persistence** or **Transient classes** – *not persistent and permanent* .
  - **Subordination** – whether they co-exist/co-persist with other classes.
  - **Distribution** – whether the operations of the objects are distributed to many nodes of the physical architecture and many processes of the process architecture.

- **Persistence**: The ability of an object to be stored and survive beyond the current program execution, typically associated with databases.
- **Transient**: Objects that exist only temporarily in memory during the current program execution and do not persist beyond it.

Architectural View(Cont'd)

Correspondence between views

From logical view to process view

- We consider each object as active, and potentially "concurrent" in logical view, ignoring the degree of concurrency.(this is why logical view concentrates on the functional requirements of the system).

From logical view to development view

- A class is usually implemented as a module.
- Large classes are decomposed into multiple packages and collections of closely related classes are grouped into subsystems.
- This is the way the development view represents the system.

Architectural View(Cont'd)

➲**From process view to physical view**

- Processes and process groups are mapped onto the available physical hardware, in various configurations for testing or deployment.
- The process view corresponds with the physical view.

Short summery

1. *The **logical view** can be used by software architect/developer to describe the functionality of the system and its components.*
2. *The **development view** can be used by software architect/developer to describe how the system is organized into modules and how they interact with each other.*
3. *The **process view** can be used by project managers to describe how the system is deployed and how it interacts with other systems.*
4. *The **physical view** can be used by system engineers to describe how the system is physically deployed and how it interacts with hardware component.*



Smile

Quiz - 3

1. What is the 4+1 Architectural Views framework?
2. Could you provide a clear description of its key elements?
3. How they collectively contribute to software architecture design?

By Aliazar D.(MSc in Software Eng.) Email :- aliazar.deneke@amu.edu.et

END

Next: Quality Attributes